

Servidor Proxy SOCKS5 con Protocolo de Monitoreo

Keoni Dubovitsky Franco Ferrari Nicolas Mazzitelli Agustin Brunero

Protocolos de Comunicación, 2026C1

Resumen

Se desarrolló un servidor proxy SOCKS5 en C11 con I/O no bloqueante multiplexado, autenticación usuario/contraseña, conexión TCP saliente a IPv4, IPv6 y FQDN, resolución DNS fuera del hilo del selector, métricas volátiles, registro de accesos y un protocolo propio de monitoreo/configuración con cliente CLI. El alcance cubre la primera entrega; el monitoreo de credenciales POP3 queda fuera.

Índice

1. Descripción de protocolos y aplicaciones	2
1.1. SOCKS5	2
1.2. Protocolo de monitoreo	2
2. Problemas encontrados	2
3. Limitaciones de la aplicación	2
4. Posibles extensiones	3
5. Conclusiones	3
6. Ejemplos de prueba	3
6.1. Corrida Pampero/Linux	4
6.2. Stress local	4
7. Guía de instalación	5
8. Instrucciones para la configuración	5
9. Ejemplos de configuración y monitoreo	6
10. Documento de diseño del proyecto	7
10.1. Estados SOCKS5	7
10.2. DNS y lifetime	7
10.3. PMC	7

1. Descripción de protocolos y aplicaciones

El proyecto entrega dos ejecutables principales. `bin/server` implementa el proxy SOCKS5 y el socket de monitoreo/configuración. `bin/client` implementa una aplicación de terminal para hablar el protocolo de monitoreo sin usar `netcat`.

1.1. SOCKS5

El servidor implementa el flujo de SOCKS5 definido por RFC1928 y RFC1929:

1. negociación de método `HELLO`;
2. autenticación usuario/contraseña;
3. `REQUEST CONNECT`;
4. resolución DNS si el destino es FQDN;
5. conexión TCP no bloqueante al origen;
6. relay bidireccional transparente entre cliente y origen.

Se soportan destinos IPv4, IPv6 y FQDN. Cuando `getaddrinfo` devuelve más de una dirección, el proxy reintenta con la siguiente dirección si el primer `connect` falla.

1.2. Protocolo de monitoreo

El Protocolo de Monitoreo y Configuración (PMC) usa TCP en un socket separado del proxy SOCKS5. Es textual, orientado a líneas CRLF, y usa respuestas `+OK/-ERR`. El cliente inicia con `HELLO 1`, luego autentica con `AUTH usuario password`.

Los comandos implementados son `ADD-USER`, `DEL-USER`, `LIST-USERS`, `METRICS`, `GET-CONFIG`, `SET-CONFIG` y `QUIT`. Las respuestas multi-línea usan prefijo de cantidad para que el cliente sepa cuántas líneas leer.

2. Problemas encontrados

Problema		Resolución
Portabilidad	Li-	Se usaron macros de plataforma en el Makefile para exponer APIs POSIX bajo C11 sin romper macOS.
nux/macOS		
I/O parcial y pipelining		Los parsers consumen desde buffers y preservan bytes sobrantes para el siguiente estado. Esto evita perder <code>AUTH</code> o <code>REQUEST</code> enviados en el mismo segmento.
Errores del selector base		Se corrigieron casos de flags no bloqueantes, límites de fd y headers no autocontenidos.
DNS bloqueante		La resolución se aisló en hilos <code>pthread</code> dedicados a <code>getaddrinfo</code> ; el resultado vuelve al selector por notificación.
Reuso de fd durante DNS		Las notificaciones de DNS llevan identidad del objeto de conexión para no despachar un resultado viejo sobre una conexión nueva.
Half-close pre-COPY		Si el cliente envía payload temprano y hace half-close mientras el <code>connect</code> está pendiente, el proxy conserva el payload y lo drena al origen al entrar en <code>COPY</code> .

3. Limitaciones de la aplicación

El selector actual usa `pselect` y queda limitado por `FD_SETSIZE`. Una conexión SOCKS5 establecida consume dos file descriptors principales: cliente-proxy y proxy-origen. Por eso el

techo real debe medirse en el entorno de entrega, no inferirse sólo desde el código.

Las métricas son volátiles: se reinician al reiniciar el servidor. El access-log actual se escribe de forma síncrona con `fprintf/fflush`; para el alcance del TP es simple y auditable, pero bajo carga extrema puede introducir latencia.

El apagado controlado tiene una decisión explícita: una segunda señal corta el drenaje graceful de conexiones activas, pero no destruye el selector ni el pool mientras haya resoluciones DNS pendientes. Esto evita use-after-free, al costo de esperar si `getaddrinfo` tarda.

El PMC no cifra el canal. Por eso se entrega con escucha por defecto en `127.0.0.1`; para una demo debe usarse `--admin` con credenciales no-default y evitar exponerlo fuera de una red administrativa controlada.

La validación con Valgrind se realizó en Pampero/Linux con tráfico real: `ERROR SUMMARY: 0 errors from 0 contexts, 0 bytes in definitely lost, indirectly lost y possibly lost`, y dos file descriptors heredados al terminar. Quedaron `5,658 bytes` como `still reachable`, atribuibles al entorno/libc y no a leaks definitivos del servidor.

4. Posibles extensiones

- migrar el selector a `poll`, `epoll` o `kqueue` para superar `FD_SETSIZE`;
- persistir métricas y rotar logs;
- agregar TLS o un esquema de autenticación más fuerte al PMC;
- exponer timeouts y más parámetros de configuración en runtime;
- exportar métricas en un formato apto para monitoreo externo;
- implementar el monitoreo de credenciales POP3 de la segunda entrega.

5. Conclusiones

La arquitectura mantiene el principio central de la consigna: I/O no bloqueante multiplexado en un único hilo, con la única excepción de hilos dedicados a DNS. La separación entre SOCKS5 y PMC permite administrar usuarios, configuración y métricas sin mezclar el plano de datos con el plano de control.

La evidencia local muestra que la suite M1-M7 pasa y que el banco de stress produce CSV y gráficos reproducibles. En Pampero/Linux se verificó la build, los unitarios, las integraciones M1-M7 y Valgrind con tráfico real sin errores ni leaks definitivos. En M5 hubo un `skip` esperado para el caso de retry multi-IP no aplicable a ese entorno, no una falla.

La prueba de stress versionada es un smoke local macOS; sirve para explicar metodología, métricas y generación de gráficos, pero no se presenta como barrido completo de capacidad en Pampero. Además se ejecutó una prueba ad hoc local de 500 conexiones con `conn_ok=500` y `conn_failed=0`.

6. Ejemplos de prueba

Comando	Estado			Observación
<code>make clean && make</code>	Verificado	local	y	Compila <code>bin/server</code> y <code>bin/client</code> sin warnings visibles.
<code>make test</code>	Verificado	local	y	Unitarios C del parser, usuarios, métricas, logger, selector bloqueante y relay.

<code>make check PORT=12080</code>	Verificado local y Pampero	Unitarios más integración M1-M7 sobre sockets reales.
<code>make valgrind PORT=...</code>	Verificado en Pampero	Valgrind con tráfico: 0 errores, 0 leaks definitivos/indirectos/posibles.
<code>test/stress/run_stress.py</code>	Smoke local macOS	Genera CSV y figuras reproducibles; no reemplaza un barrido completo de capacidad en Pampero.

6.1. Corrida Pampero/Linux

La corrida remota se ejecutó en `pampero.it.itba.edu.ar`, Linux 7.0.9-arch1-1, GCC 16.1.1 20260625. El build `make clean && make` terminó con BUILD OK.

Los unitarios reportaron todos los casos OK; entre los totales relevantes:

- `hello_test` 21/0, `auth_test` 19/0, `users_test` 35/0 y `request_test` 63/0;
- `connect_test` 23/0, `copy_test` 18/0 y `netutils_test` 20/0;
- `selector_block_test` 17/0, `metrics_test` 16/0 y `logger_test` 4/0.

La integración M1-M7 terminó sin fallas:

- M1: 6 ok, 0 fallas;
- M2: 8 ok, 0 fallas;
- M3: 14 ok, 0 fallas;
- M4: 19 ok, 0 fallas;
- M5: 7 ok, 0 fallas, 1 skip; el skip corresponde al retry multi-IP no aplicable en ese entorno, no a una falla funcional;
- M6: access-log OK y FAIL emitidos;
- M7: 35 ok, 0 fallas.

Valgrind con tráfico real reportó:

- FILE DESCRIPTORS: 2 open (2 inherited) at exit;
- definitely lost: 0 bytes, indirectly lost: 0 bytes y possibly lost: 0 bytes;
- ERROR SUMMARY: 0 errors from 0 contexts.

Los 5,658 bytes still reachable se consideran alcanzables del entorno/libc, no leaks definitivos del programa.

6.2. Stress local

El banco de stress se genera con `test/stress/run_stress.py`. El script levanta un origen controlado, ejecuta el servidor SOCKS5, consulta métricas por PMC y escribe CSV bajo un directorio de salida. Los parámetros principales son:

- concurrencia: cantidad de conexiones simultáneas por caso;
- payload: bytes transferidos por conexión;
- modo de destino: `ipv4` o `fqdn`;
- métricas recolectadas: conexiones OK/fallidas, bytes, wall time, throughput, tasa de conexiones, latencias p50/p95, REP no exitosos y deltas de métricas PMC.

La evidencia versionada corresponde al smoke local macOS guardado en:

`docs/stress/2026-07-05_local-smoke_v1/`

Ese smoke usó concurrencias 5 y 20, payloads 1024 y 8192 bytes, targets IPv4 y FQDN, y una repetición por combinación. También hubo una prueba local ad hoc de 500 conexiones con `conn_ok=500` y `conn_failed=0`. No se presenta como máximo absoluto del servidor ni como barrido completo en Pampero.

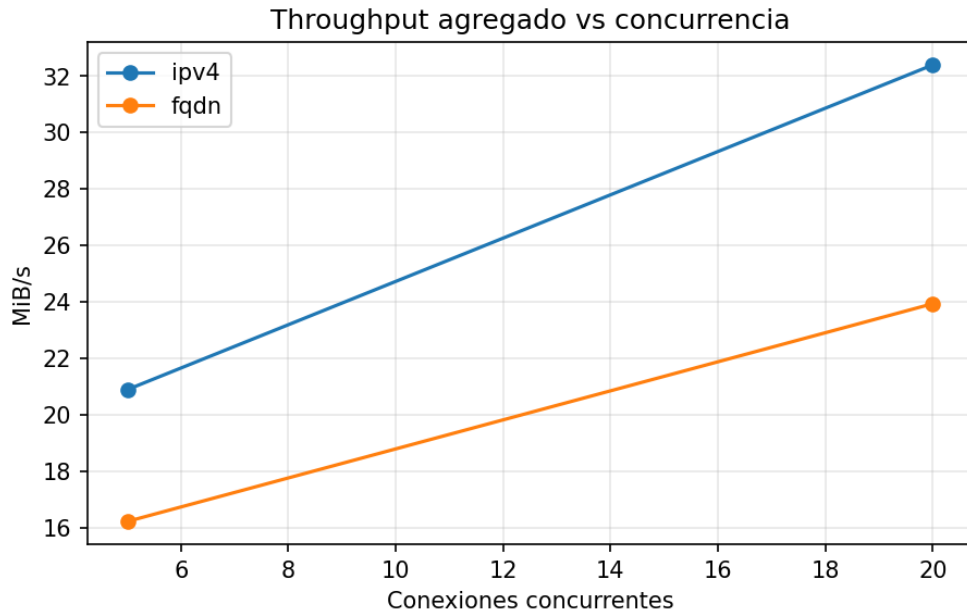


Figura 1: Throughput agregado del smoke local por concurrencia y modo de destino.

7. Guía de instalación

Dependencias:

- compilador C11 (gcc o clang);
- make;
- python3 para integraciones y stress;
- valgrind en Linux/Pampero para memoria/fds;
- matplotlib si se quieren regenerar gráficos de stress;
- rsync y ssh para el runner de Pampero.

Compilación:

```
make clean && make
```

Artefactos generados:

- bin/server: servidor SOCKS5 + PMC;
- bin/client: cliente CLI del PMC;
- docs/report/main.pdf: este informe.

Verificación remota recomendada:

```
PAMPERO_USER=<usuario> bash scripts/run-on-pampero.sh 12080
```

El script sube el repo a Pampero, recompila desde cero y ejecuta unitarios, integración M1-M7 y Valgrind con tráfico real.

8. Instrucciones para la configuración

Opciones principales del servidor:

```
./bin/server [OPCIONES]
-l <addr>          direccion SOCKS, default 0.0.0.0
-p <port>          puerto SOCKS, default 1080
-L <addr>          direccion PMC, default 127.0.0.1
```

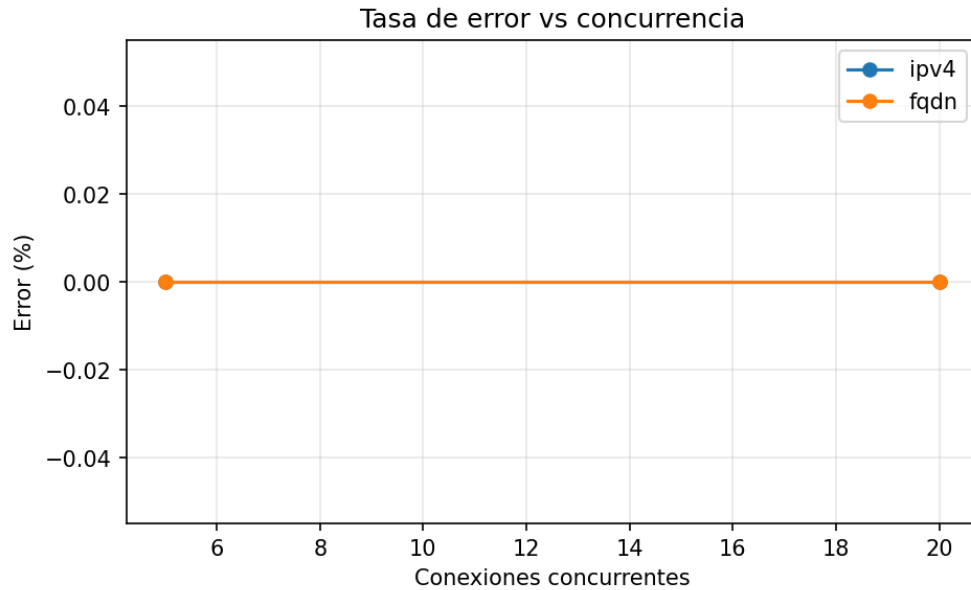


Figura 2: Tasa de error del smoke local. En la corrida versionada no hubo conexiones fallidas.

```
-P <port>          puerto PMC, default 8080
-u <name>:<pass>   usuario SOCKS5
--admin u:p        credencial de administracion PMC
-N                desactiva disectores
-h / -v           ayuda / version
```

El tamaño de buffer se puede consultar y cambiar para conexiones nuevas mediante el PMC con `GET-CONFIG buffer-size` y `SET-CONFIG buffer-size N`.

9. Ejemplos de configuración y monitoreo

Ejemplos con el cliente:

```
./bin/client --admin admin:s3cr3t metrics
./bin/client --admin admin:s3cr3t add-user pablito pass1234
./bin/client --admin admin:s3cr3t list-users
./bin/client --admin admin:s3cr3t get-config buffer-size
./bin/client --admin admin:s3cr3t set-config buffer-size 32768
./bin/client --admin admin:s3cr3t del-user pablito
./bin/client --admin admin:s3cr3t quit
```

Ejemplo de sesión cruda PMC:

```
C: HELLO 1
S: +OK 1
C: AUTH admin s3cr3t
S: +OK
C: METRICS
S: +OK 5
S: historic-connections 10
S: concurrent-connections 0
S: bytes-transferred 123456
S: configured-users 2
```

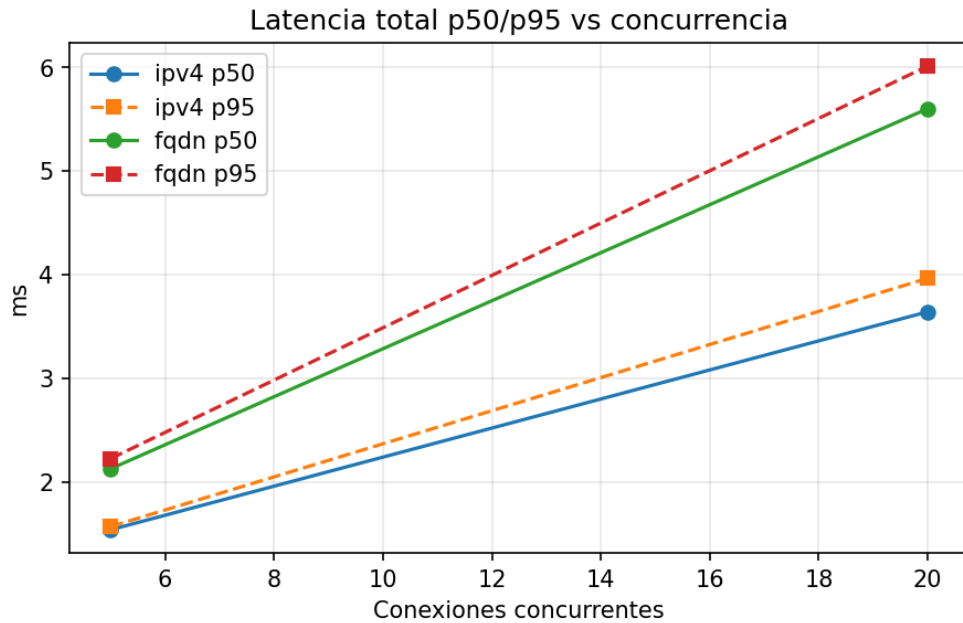


Figura 3: Latencias p50/p95 totales del smoke local.

```
S: failed-connections 1
C: QUIT
S: +OK bye
```

10. Documento de diseño del proyecto

El servidor usa tres piezas centrales del toolkit: **selector**, **stm** y **buffer**. El selector concentra los eventos de sockets; la máquina de estados separa cada fase del protocolo; los buffers resuelven lecturas y escrituras parciales.

10.1. Estados SOCKS5

```
HELLO ->AUTH ->REQUEST ->RESOLV? ->CONNECTING ->REQUEST_WRITE ->COPY ->DONE
```

COPY mantiene dos direcciones de relay: cliente a origen y origen a cliente. Cada dirección tiene su estado de lectura/escritura; al recibir EOF en un sentido se drena lo pendiente y se propaga `shutdown(SHUT_WR)` al par.

10.2. DNS y lifetime

La resolución FQDN corre en un hilo detached que sólo ejecuta `getaddrinfo` y notifica al selector. La notificación lleva identidad del objeto de conexión y cleanup asociado. El proceso de apagado espera a que no haya resoluciones DNS en vuelo antes de destruir selector y pool.

10.3. PMC

El PMC tiene su propio listener y su propia STM, pero comparte el mismo selector del servidor. El cliente CLI usa I/O bloqueante por simplicidad, permitido por la consigna, y encapsula el framing del protocolo.